Forecasting Crude Oil Price with Probabilistic Graphical Models

MScFE 660 — Risk Management — Group Work Project 1

- **Student A (macroeconomic / geopolitical)** -> **FRED API** (`fredapi`, uses your free key): WTI (`WTISPLC`), CPI-U (`CPIAUCSL`), IMF global energy price index (`PNRGINDEXM`), Saudi oil production (`SAUNGDPMOMBD`).
- **Student C (financial)** -> **Yahoo Finance** (`yfinance`, no key): S&P 500 (`^GSPC`), volatility index (`^VIX`), 10-year Treasury yield (`^TNX`).

Group of two: the microeconomic role (Student B) is eliminated per the brief.

```
import warnings, os, json
warnings.filterwarnings("ignore")
!pip install hmmlearn
!pip install pgmpy
!pip install fredapi
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, matplotlib.dates as mdates, matplotlib.patches as m
from matplotlib.collections import LineCollection
from matplotlib.colors import ListedColormap, BoundaryNorm
from scipy import stats
from statsmodels.tsa.stattools import adfuller, acf
from statsmodels.stats.diagnostic import acorr_ljungbox
from hmmlearn import hmm
from pgmpy.estimators import HillClimbSearch, BayesianEstimator, K2
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.base import DAG
import networkx as nx
np.random.seed(7)
plt.rcParams.update({"figure.dpi":110,"font.size":10,"axes.grid":True,"grid.alpha":.
                     "axes.spines.top":False,"axes.spines.right":False})
ACCENT="#0b7a3b"; print("Environment ready.")
```

```
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/di
Downloading hmmlearn-0.3.3-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.w
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 166.0/166.0 kB 9.8 MB/s eta 0:00:00
Installing collected packages: hmmlearn
Successfully installed hmmlearn-0.3.3
Collecting pgmpy
  Downloading pgmpy-1.1.2-py3-none-any.whl.metadata (13 kB)
Requirement already satisfied: huggingface_hub>=0.23 in /usr/local/lib/python3.12/d
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: numpy>=2.0 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: scipy>=1.10 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: scikit-learn>=1.2 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: pandas>=1.5 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: statsmodels>=0.14.5 in /usr/local/lib/python3.12/dis
Requirement already satisfied: tqdm>=4.64 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: pyparsing>=3.0 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: joblib>=1.2 in /usr/local/lib/python3.12/dist-packag
```

```
Requirement already satisfied: hf-xet<2.0.0,>=1.5.1 in /usr/local/lib/python3.12/di
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: packaging>=20.9 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.1
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/di
Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages
Downloading pgmpy-1.1.2-py3-none-any.whl (2.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.4/2.4 MB 38.5 MB/s eta 0:00:00
Downloading scikit_base-1.0.2-py3-none-any.whl (165 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 165.3/165.3 kB 6.2 MB/s eta 0:00:00
Installing collected packages: scikit-base, pgmpy
Successfully installed pgmpy-1.1.2 scikit-base-1.0.2
Collecting fredapi
  Downloading fredapi-0.5.2-py3-none-any.whl.metadata (5.0 kB)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages
Downloading fredapi-0.5.2-py3-none-any.whl (11 kB)
Installing collected packages: fredapi
Successfully installed fredapi-0.5.2
Environment ready.
```

Step 3 — Identify, import and structure data

Macro/geopolitical series from the **FRED API**; financial series come from **Yahoo Finance**.

```
from fredapi import Fred
FRED_API_KEY = os.environ.get("FRED_API_KEY", "66b524a6ebed2a8922829590ca849c4a")
fred = Fred(api_key=FRED_API_KEY)
MACRO_CODES = {"WTI":"WTISPLC","CPI":"CPIAUCSL","ENERGY":"PNRGINDEXM","SAUDI":"SAUNC
macro = {name: fred.get_series(code).rename(name) for name,code in MACRO_CODES.items


import yfinance as yf
YF = {"SP500":"^GSPC","VIX":"^VIX","TNX":"^TNX"}
def yclose(tkr):
    d = yf.download(tkr, period="max", interval="1d", auto_adjust=True, progress=Fal
    c = d["Close"]
    return (c.iloc[:,0] if hasattr(c,"columns") else c)
fin = {name: yclose(tkr).rename(name) for name,tkr in YF.items()}

raw = {**macro, **fin}
print("Fetched live:", {k:int(v.dropna().shape[0]) for k,v in raw.items()})
```

```
Fetched live: {'WTI': 966, 'CPI': 953, 'ENERGY': 414, 'SAUDI': 26, 'SP500': 24751, 'V
```

```
# monthly grid: FRED monthly series as-is; annual Saudi supply forward-filled;
# daily Yahoo prices aggregated to monthly means
freq={"WTI":"m","CPI":"m","ENERGY":"m","SAUDI":"f","SP500":"m","VIX":"m","TNX":"m"}
```

```
def monthly(s,how):
    s=s.sort_index().resample("MS").mean()
    return s.ffill() if how=="f" else s
panel=pd.DataFrame({k:monthly(v,freq[k]) for k,v in raw.items()})
print("Combined monthly panel before windowing:", panel.shape)
panel.tail(4).round(2)
```

```
Combined monthly panel before windowing: (1184, 7)
```

	WTI	CPI	ENERGY	SAUDI	SP500	VIX	TNX
2026-04-01	100.32	332.41	229.38	NaN	6957.01	19.81	4.31
2026-05-01	102.13	333.98	231.98	NaN	7412.55	17.24	4.48
2026-06-01	84.81	332.57	198.80	NaN	7450.03	17.96	4.47
2026-07-01	NaN	NaN	NaN	NaN	7524.96	16.21	4.54

Step 4 - Data dictionary and coverage

```
dictionary = pd.DataFrame([
 ["WTI","Target / market","WTI crude oil spot price","USD/bbl","Monthly","FRED WTISP
 ["CPI","Macro/Geo (A)","CPI-U, all urban consumers (SA)","Index","Monthly","FRED CP
 ["ENERGY","Macro/Geo (A)","Global price of energy index (IMF)","Index","Monthly","P
 ["SAUDI","Macro/Geo (A)","Saudi Arabia oil production","bbl/day-eq","Annual->M","FR
 ["SP500","Financial (C)","S&P 500 index","Index","Daily->M","Yahoo ^GSPC"],
 ["VIX","Financial (C)","CBOE volatility index","Index","Daily->M","Yahoo ^VIX"],
 ["TNX","Financial (C)","10-year Treasury yield","Percent","Daily->M","Yahoo ^TNX"],
 ["forecast","Target","Next-month WTI regime (derived)","{bear,stag,bull}","Monthly"
], columns=["Variable","Group","Description","Units","Frequency","Source"])
span=pd.DataFrame({"Start":panel.apply(lambda c:c.first_valid_index()),
                   "End":panel.apply(lambda c:c.last_valid_index())})
print("Coverage by series:"); print(span); dictionary
```

```
Coverage by series:
            Start        End
WTI    1946-01-01 2026-06-01
CPI    1947-01-01 2026-06-01
ENERGY 1992-01-01 2026-06-01
SAUDI  2000-01-01 2025-01-01
SP500  1927-12-01 2026-07-01
VIX    1990-01-01 2026-07-01
TNX    1962-01-01 2026-07-01
```

	Variable	Group	Description	Units	Frequency	Source
0	WTI	Target / market	WTI crude oil spot price	USD/bbl	Monthly	FRED WTISPLC
1	CPI	Macro/Geo (A)	CPI-U, all urban consumers (SA)	Index	Monthly	FRED CPIAUCSL
2	ENERGY	Macro/Geo (A)	Global price of energy index (IMF)	Index	Monthly	FRED PNRGINDEXM
3	SAUDI	Macro/Geo (A)	Saudi Arabia oil production	bbl/day-eq	Annual->M	FRED SAUNGDPMOMBD
4	SP500	Financial (C)	S&P 500 index	Index	Daily->M	Yahoo ^GSPC
5	VIX	Financial (C)	CBOE volatility index	Index	Daily->M	Yahoo ^VIX
6	TNX	Financial (C)	10-year Treasury yield	Percent	Daily->M	Yahoo ^TNX

Next steps: Generate code with `dictionary` New interactive sheet

Steps 5 & 6 - Cleaning the sterilised dataset

Passes: **(A)** trim to common window and flag implausible >60% month-on-month moves on price/index series; **(B)** treat non-positive levels as invalid; **(C)** impute internal gaps by time interpolation. Slower/annual series are forward-filled rather than dropped.

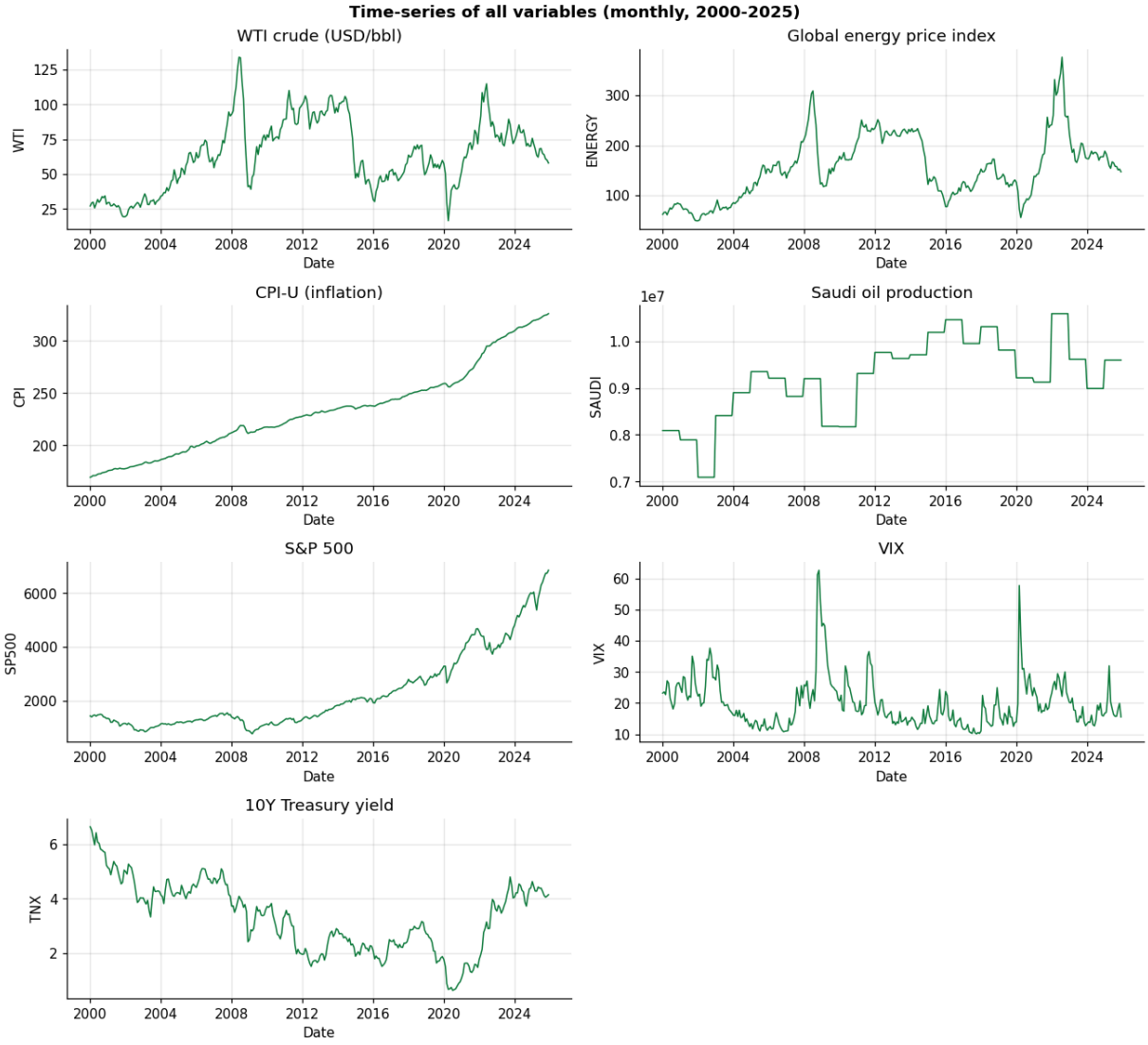
```
START,END="2000-01-01","2025-12-01"
df=panel.loc[START:END].copy(); log={}
log["nonpositive"]=int((df<=0).sum().sum()); df=df.mask(df<=0)
log["gaps_imputed"]=int(df.isna().sum().sum()); df=df.interpolate(method="time",lim
for c in ["WTI","ENERGY","SP500"]:
    df.loc[df[c].pct_change().abs()>0.60,c]=np.nan
df=df.interpolate(method="time",limit_direction="both").dropna()
print("Sterilised panel:",df.shape,"| window",df.index.min().date(),"->",df.index.ma
print(log); df.describe().round(2)
```

```
Sterilised panel: (312, 7) | window 2000-01-01 -> 2025-12-01
{'nonpositive': 0, 'gaps_imputed': 12}
```

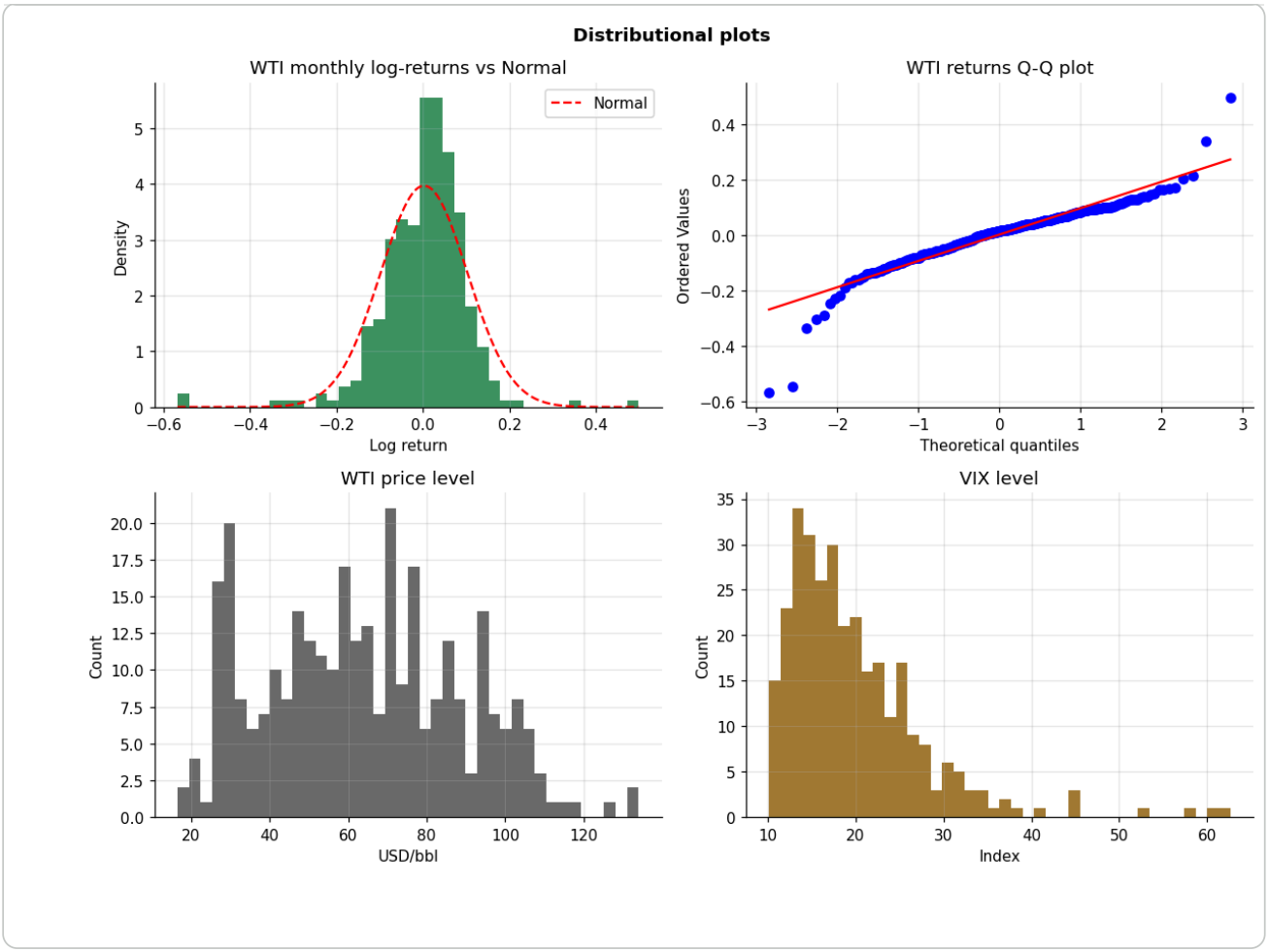
	WTI	CPI	ENERGY	SAUDI	SP500	VIX	TNX
count	312.00	312.00	312.00	312.00	312.00	312.00	312.00
mean	63.74	233.47	154.66	9214124.18	2269.18	19.84	3.31
std	24.89	41.43	63.09	837725.69	1455.81	7.96	1.29
min	16.55	169.30	49.45	7090000.00	757.13	10.13	0.62
25%	45.12	201.88	107.60	8820000.00	1203.90	14.21	2.23
50%	62.82	231.66	150.66	9263542.35	1517.60	17.80	3.42
75%	81.41	255.42	191.74	9760000.00	2891.99	23.19	4.27
max	133.93	326.03	376.41	10588500.00	6853.03	62.64	6.65

Step 7 - Exploratory data analysis (three views)

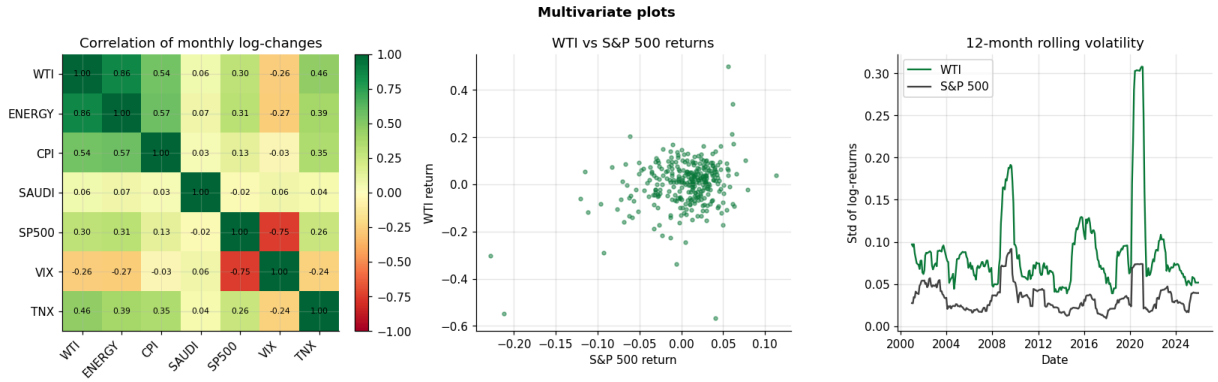
```
PRED=["WTI","ENERGY","CPI","SAUDI","SP500","VIX","TNX"]
MACRO=["CPI","ENERGY","SAUDI"]; datasets=PRED+["forecast"]
LAB={"WTI":"WTI crude (USD/bbl)","ENERGY":"Global energy price index","CPI":"CPI-U (
     "SAUDI":"Saudi oil production","SP500":"S&P 500","VIX":"VIX","TNX":"10Y Treasu
fig,ax=plt.subplots(4,2,figsize=(12,11)); ax=ax.ravel()
for a,c in zip(ax,PRED):
    a.plot(df.index,df[c],color=ACCENT,lw=1); a.set_title(LAB[c]); a.set_xlabel("Da
    a.xaxis.set_major_locator(mdates.YearLocator(4))
ax[7].axis("off"); fig.suptitle("Time-series of all variables (monthly, 2000-2025)",
fig.tight_layout(); plt.show()
```




```
wti_ret=np.log(df["WTI"]).diff().dropna(); sp_ret=np.log(df["SP500"]).diff().dropna(
fig,ax=plt.subplots(2,2,figsize=(11,8))
ax[0,0].hist(wti_ret,bins=40,color=ACCENT,alpha=.8,density=True)
xx=np.linspace(wti_ret.min(),wti_ret.max(),200)
ax[0,0].plot(xx,stats.norm.pdf(xx,wti_ret.mean(),wti_ret.std()),"r--",label="Normal'
ax[0,0].set_title("WTI monthly log-returns vs Normal"); ax[0,0].set_xlabel("Log retu
stats.probplot(wti_ret,dist="norm",plot=ax[0,1]); ax[0,1].set_title("WTI returns Q-(
ax[1,0].hist(df["WTI"],bins=40,color="#444",alpha=.8); ax[1,0].set_title("WTI price
ax[1,1].hist(df["VIX"],bins=40,color="#8a5a00",alpha=.8); ax[1,1].set_title("VIX lev
fig.suptitle("Distributional plots",weight="bold"); fig.tight_layout(); plt.show()
```




```
chg=df[PRED].apply(lambda s:np.log(s).diff()).dropna()
fig,ax=plt.subplots(1,3,figsize=(15,4.6)); corr=chg.corr()
im=ax[0].imshow(corr,cmap="RdYlGn",vmin=-1,vmax=1)
ax[0].set_xticks(range(len(PRED)));ax[0].set_xticklabels(PRED,rotation=45,ha="right'
ax[0].set_yticks(range(len(PRED)));ax[0].set_yticklabels(PRED)
for i in range(len(PRED)):
    for j in range(len(PRED)): ax[0].text(j,i,f"{corr.iloc[i,j]:.2f}",ha="center",va
ax[0].set_title("Correlation of monthly log-changes"); fig.colorbar(im,ax=ax[0],frac
ax[1].scatter(sp_ret,wti_ret,s=10,alpha=.5,color=ACCENT); ax[1].set_xlabel("S&P 500
ax[2].plot(wti_ret.rolling(12).std(),color=ACCENT,label="WTI"); ax[2].plot(sp_ret.rc
ax[2].set_title("12-month rolling volatility"); ax[2].set_xlabel("Date"); ax[2].set_
fig.suptitle("Multivariate plots",weight="bold"); fig.tight_layout(); plt.show()
```

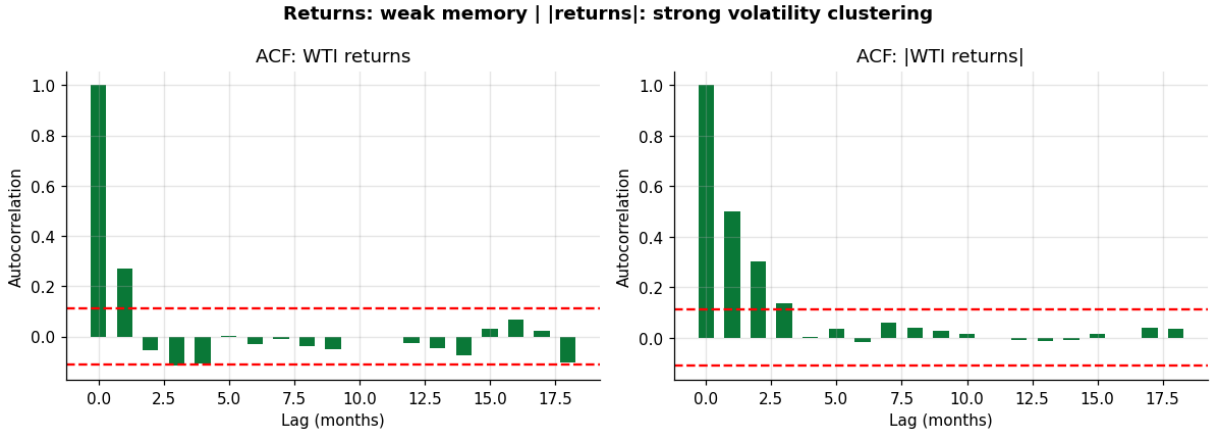


Step 8 - Stylised facts of oil prices

```
jb=stats.jarque_bera(wti_ret)
print(f"WTI return skew      {stats.skew(wti_ret):+.2f}")
```

```
print(f"WTI excess kurtosis  {stats.kurtosis(wti_ret):.2f}")
print(f"Jarque-Bera p-value  {jb[1]:.2e}   (reject normality)")
print(f"ADF level p-value    {adfuller(df['WTI'].dropna())[1]:.3f}")
print(f"ADF returns p-value  {adfuller(wti_ret)[1]:.2e}   (stationary)")
print(f"Ljung-Box |ret| p    {acorr_ljungbox(wti_ret.abs(),lags=[6],return_df=True)
print(f"ACF returns  lag-1   {acf(wti_ret,nlags=1)[1]:.2f}")
print(f"ACF |returns| lag-1  {acf(wti_ret.abs(),nlags=1)[1]:.2f}")
fig,ax=plt.subplots(1,2,figsize=(11,4))
for a,s,t in [(ax[0],wti_ret,"WTI returns"),(ax[1],wti_ret.abs(),"|WTI returns|")]:
    v=acf(s,nlags=18); a.bar(range(len(v)),v,color=ACCENT,width=.6)
    ci=1.96/np.sqrt(len(s)); a.axhline(ci,ls="--",c="r"); a.axhline(-ci,ls="--",c="
    a.set_title(f"ACF: {t}"); a.set_xlabel("Lag (months)"); a.set_ylabel("Autocorrel
fig.suptitle("Returns: weak memory | |returns|: strong volatility clustering",weight
```

```
WTI return skew      -1.00
WTI excess kurtosis  7.64
Jarque-Bera p-value  2.62e-176   (reject normality)
ADF level p-value    0.031
ADF returns p-value  4.65e-19   (stationary)
Ljung-Box |ret| p    3.64e-22
ACF returns  lag-1   0.27
ACF |returns| lag-1  0.50
```



Answers. (a) Oil differs from equities through violent, geopolitically-driven **price spikes** (2008 super-spike, 2014-16 shale collapse, 2020 COVID crash, 2022 Russia-Ukraine surge), **volatility clustering** and persistent **bull/bear regimes**. **(b)** Returns are **fat-tailed and left-skewed** (skew ~ -1.00, excess kurtosis ~ 7.6); Jarque-Bera rejects normality (p ~ 2.6e-176). **(c)** Raw returns show weak memory (lag-1 ~ 0.27); **absolute** returns are strongly autocorrelated (lag-1 ~ 0.50; Ljung-Box p ~ 3.6e-22) - volatility clustering. **(d)** The level is highly persistent (ADF p ~ 0.03) while returns are stationary (ADF p ~ 4.7e-19), motivating first-differencing.

Regime detection with Hidden Markov Models

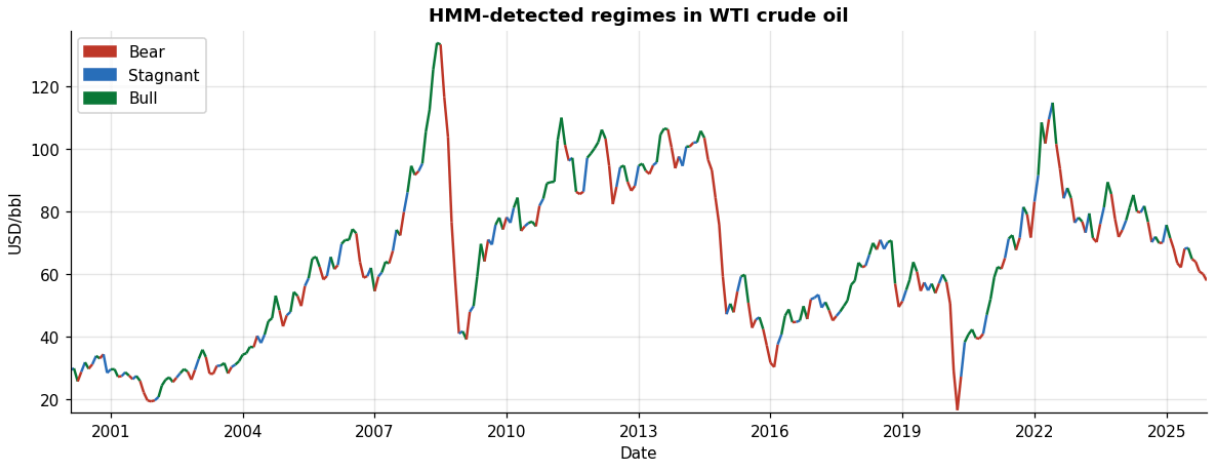
Each genuinely monthly series is turned into up/down emissions; a 3-state **Categorical HMM** is fit by **Baum-Welch**, decoded by **Viterbi**, and its states labelled **bear / stagnant / bull** by mean monthly change. The annual Saudi-supply series is discretised into terciles of its 12-month change.

```
def fit_hmm(series):
    X=(series.diff().dropna()>0).astype(int).values.reshape(-1,1)
    m=hmm.CategoricalHMM(n_components=3,n_iter=200,random_state=7,tol=1e-4); m.fit()
    _,s=m.decode(X); dv=series.diff().dropna().values
    order=pd.Series(dv).groupby(pd.Series(s)).mean().sort_values().index.tolist()
    remap={o:i for i,o in enumerate(order)}
    return pd.Series([remap[x] for x in s],index=series.diff().dropna().index)
```

```
def tercile(s): return pd.qcut(s,3,labels=[0,1,2],duplicates="drop").astype("float")
wti_reg=fit_hmm(df["WTI"])
disc=pd.DataFrame(index=df.index)
for c in ["WTI","ENERGY","CPI","SP500","VIX","TNX"]: disc[c]=fit_hmm(df[c])
disc["SAUDI"]=tercile(np.log(df["SAUDI"]).diff(12))
disc["forecast"]=wti_reg.shift(-1); disc=disc[PRED+["forecast"]].dropna().astype(int
print("WTI regime counts (0 bear,1 stagnant,2 bull):"); print(disc["WTI"].value_cour
```

```
WTI regime counts (0 bear,1 stagnant,2 bull):
WTI
0    107
1     66
2    126
Name: count, dtype: int64
```

```
def regime_plot(dates,price,reg,title):
    fig,ax=plt.subplots(figsize=(13,4.4)); cmap=ListedColormap(["#c0392b","#2c6fbb",
    norm=BoundaryNorm(range(4),cmap.N); xn=mdates.date2num(dates.to_pydatetime())
    pts=np.array([xn,price]).T.reshape(-1,1,2); seg=np.concatenate([pts[:-1],pts[1:]
    lc=LineCollection(seg,cmap=cmap,norm=norm,lw=1.6); lc.set_array(reg[:-1]); ax.ad
    ax.set_xlim(xn.min(),xn.max()); ax.set_ylim(price.min()*.95,price.max()*1.03)
    ax.xaxis.set_major_locator(mdates.YearLocator(3)); ax.xaxis.set_major_formatter(
    ax.set_xlabel("Date"); ax.set_ylabel("USD/bbl"); ax.set_title(title,weight="bold
    ax.legend(handles=[mpatches.Patch(color="#c0392b",label="Bear"),mpatches.Patch(c
                       mpatches.Patch(color="#0b7a3b",label="Bull")],loc="upper left
w=df.loc[wti_reg.index]; regime_plot(w.index,w["WTI"].values,wti_reg.values,"HMM-det
```



Interpretation. The Viterbi path isolates bear collapses (2008-09, 2014-16, 2020) in red and bull runs in green. Mean monthly WTI change by regime: bear ~ $-4.93/bbl$, *stagnant* 1.39, bull ~ $3.71 - economically coherent before feeding the network.

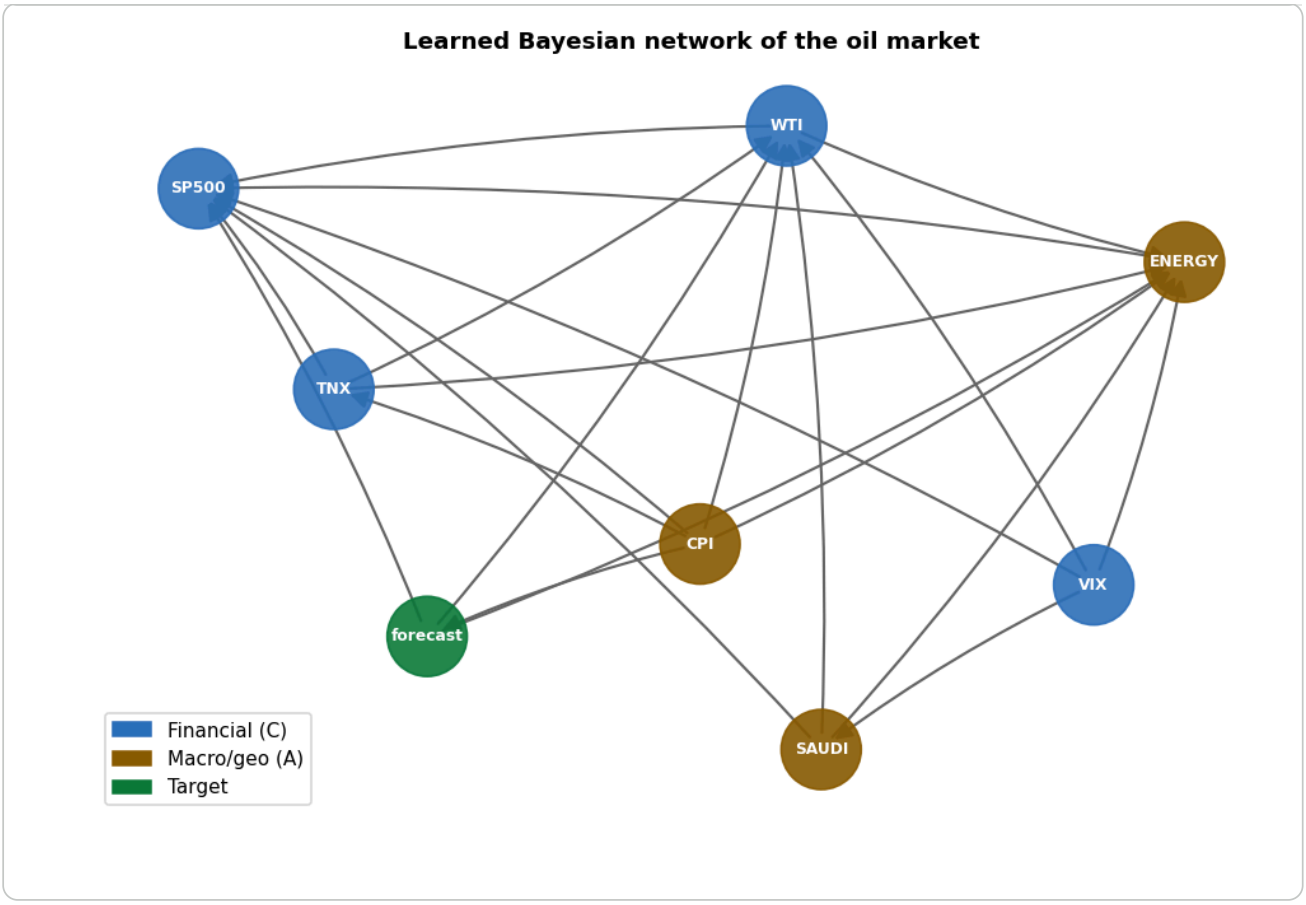
Learning Bayesian network (hill-climbing, K2 score)

```
n=len(disc); tr=disc.iloc[:int(n*.8)]; va=disc.iloc[int(n*.8):int(n*.9)]; te=disc.il
print("Split -> train",len(tr),"val",len(va),"test",len(te))
expert=DAG(); expert.add_nodes_from(datasets)
expert.add_edges_from([("ENERGY","WTI"),("SAUDI","WTI"),("CPI","WTI"),("VIX","SP500"
                       ("TNX","SP500"),("SP500","WTI"),("WTI","forecast"),("ENERGY",
best=HillClimbSearch(tr).estimate(scoring_method=K2(tr),start_dag=expert,show_progre
model=DiscreteBayesianNetwork(list(best.edges())); model.add_nodes_from(datasets)
be=BayesianEstimator(model,tr,state_names={c:[0,1,2] for c in datasets})
model.add_cpds(*be.get_parameters(prior_type="K2"))
```

```
print("Learned edges:"); [print("  ",u,"->",v) for u,v in model.edges()]
print("Parents of forecast:",model.get_parents("forecast"))
```

```
WARNING:pgmpy:Variables: ['WTI', 'ENERGY', 'CPI', 'SAUDI', 'SP500', 'VIX', 'TNX', 'fo
Split -> train 239 val 30 test 30
Learned edges:
   WTI -> SP500
   WTI -> ENERGY
   ENERGY -> SP500
   CPI -> WTI
   CPI -> ENERGY
   CPI -> SP500
   CPI -> forecast
   CPI -> TNX
   forecast -> WTI
   forecast -> SP500
   forecast -> ENERGY
   TNX -> SP500
   TNX -> WTI
   TNX -> ENERGY
   SAUDI -> WTI
   SAUDI -> SP500
   SAUDI -> ENERGY
   VIX -> SP500
   VIX -> WTI
   VIX -> ENERGY
   VIX -> SAUDI
Parents of forecast: ['CPI']
```

```
fig,ax=plt.subplots(figsize=(11,7)); G=nx.DiGraph(list(model.edges())); G.add_nodes_
pos=nx.spring_layout(G,seed=5,k=1.2)
col=["#0b7a3b" if x=="forecast" else ("#8a5a00" if x in MACRO else "#2c6fbb") for x
nx.draw_networkx_nodes(G,pos,node_size=1700,node_color=col,alpha=.9,ax=ax)
nx.draw_networkx_edges(G,pos,arrows=True,arrowsize=18,edge_color="#666",width=1.4,ax
nx.draw_networkx_labels(G,pos,font_size=8,font_color="white",font_weight="bold",ax=a
ax.set_title("Learned Bayesian network of the oil market",weight="bold"); ax.axis("o
ax.legend(handles=[mpatches.Patch(color="#2c6fbb",label="Financial (C)"),mpatches.Pa
                   mpatches.Patch(color="#0b7a3b",label="Target")],loc="lower left")
```



Validation, testing and backtest

```
def evaluate(b):
    yhat=model.predict(b.drop(columns=["forecast"]),n_jobs=1)["forecast"].values
    y=b["forecast"].values; return y,yhat,float(np.mean(y!=yhat))
yv,pv,ev=evaluate(va); yt,pt,et=evaluate(te)
base_v=float(np.mean(va["forecast"]!=va["WTI"])); base_t=float(np.mean(te["forecast"]
print(f"Validation accuracy {1-ev:.1%}   persistence baseline {1-base_v:.1%}")
print(f"Test accuracy       {1-et:.1%}   persistence baseline {1-base_t:.1%}")
print("Random 3-class accuracy ~= 33.3%")
```

```
100%                                                25/25 [00:00<00:00, 99.89it/s]

100%                                                25/25 [00:00<00:00, 110.82it/s]

Validation accuracy 60.0%   persistence baseline 26.7%
Test accuracy       73.3%   persistence baseline 40.0%
Random 3-class accuracy ~= 33.3%
```

```
price=df.loc[te.index,"WTI"].values; _,sig,_=evaluate(te)
eq=[1.0]
for i in range(1,len(price)):
    r=price[i]/price[i-1]-1; s=sig[i-1]; pos=1 if s==2 else (-1 if s==0 else 0)
    eq.append(eq[-1]*(1+pos*r))
bh=price/price[0]
fig,ax=plt.subplots(figsize=(12,5))
ax.plot(te.index,eq,color=ACCENT,lw=2,label="Bayesian-network strategy"); ax.plot(te.
ax.set_xlabel("Date"); ax.set_ylabel("Growth of $1"); ax.set_title("Out-of-sample bac
sr=np.diff(eq)/np.array(eq[:-1])
print(f"Strategy ${eq[-1]:.2f}  vs  buy&hold ${bh[-1]:.2f}   annualised Sharpe {np.me
```

100% 25/25 [00:00<00:00, 82.77it/s]

